

React Context API Hands-On Lab — Solution

Converting the Employee Management App's Theme Prop-Drilling to React Context

Objectives

- Explain the need and benefits of the React Context API
- Work with createContext()
- List the types of Router components

In this hands-on lab, you will learn how to:

- Create a context to be used by child components
- Create a provider and a consumer of the context

Prerequisites

- Node.js
- NPM
- Visual Studio Code

Task

The employee management application currently passes the light/dark theme name from App down to EmployeesList and then down to EmployeeCard using props ("prop drilling"). Convert this so that the theme name is shared using the React Context API instead, removing the need to pass it explicitly through EmployeesList.

Solution — Step by Step

Step 1 & 2: Unzip the application and restore packages

Unzip the application, open it in VS Code, then restore the node modules from the terminal:

```
npm install
```

Step 3: Run the application to see the current output

```
npm start
```

The app opens at <http://localhost:3000> and shows the employee list with buttons styled according to the current theme (light or dark), which is currently passed via props.

Step 4: Explore the existing components

Before making changes, review how the theme currently flows through the three files. A typical props-based implementation looks like this:

App.js (before):

```
import React, { Component } from 'react';
import EmployeesList from './EmployeesList';
```

```

class App extends Component {
  constructor(props) {
    super(props);
    this.state = { theme: 'light' };
  }

  toggleTheme = () => {
    this.setState(prevState => ({
      theme: prevState.theme === 'light' ? 'dark' : 'light'
    }));
  };

  render() {
    return (
      <div className="App">
        <button onClick={this.toggleTheme}>Toggle Theme</button>
        <EmployeesList theme={this.state.theme} />
      </div>
    );
  }
}

export default App;

```

EmployeesList.js (before):

```

import React from 'react';
import EmployeeCard from './EmployeeCard';
import employees from './EmployeesMock';

function EmployeesList({ theme }) {
  return (
    <div className="employees-list">
      {employees.map(emp => (
        <EmployeeCard key={emp.id} employee={emp} theme={theme} />
      ))}
    </div>
  );
}

export default EmployeesList;

```

EmployeeCard.js (before):

```

import React from 'react';

function EmployeeCard({ employee, theme }) {
  return (
    <div className="employee-card">
      <h3>{employee.name}</h3>
      <p>{employee.designation}</p>
    </div>
  );
}

```

```

        <button className={theme === 'dark' ? 'btn-dark' : 'btn-light'}>
          View Details
        </button>
      </div>
    );
  }

  export default EmployeeCard;

```

Step 5: Create ThemeContext.js

Add a new file named ThemeContext.js in the src folder that defines the context with a default value of 'light' and exports it as the default export:

```

import { createContext } from 'react';

const ThemeContext = createContext('light');

export default ThemeContext;

```

Step 6: Update App.js to provide the context

a. Import ThemeContext into App.js:

```

import ThemeContext from './ThemeContext';

```

b, c & d. Wrap the entire JSX of the App component in ThemeContext.Provider, set its value from the component's state, and stop passing theme as a prop to EmployeesList:

```

import React, { Component } from 'react';
import EmployeesList from './EmployeesList';
import ThemeContext from './ThemeContext';

class App extends Component {
  constructor(props) {
    super(props);
    this.state = { theme: 'light' };
  }

  toggleTheme = () => {
    this.setState(prevState => ({
      theme: prevState.theme === 'light' ? 'dark' : 'light'
    }));
  };

  render() {
    return (
      <ThemeContext.Provider value={this.state.theme}>
        <div className="App">
          <button onClick={this.toggleTheme}>Toggle Theme</button>
          <EmployeesList />
        </div>
      </ThemeContext.Provider>
    );
  }
}

```

```

        </div>
      </ThemeProvider>
    );
  }
}

export default App;

```

Step 7: Update EmployeesList.js to stop passing theme down

Since EmployeeCard will read the theme from context directly, EmployeesList no longer needs to accept or forward a theme prop:

```

import React from 'react';
import EmployeeCard from './EmployeeCard';
import employees from './EmployeesMock';

function EmployeesList() {
  return (
    <div className="employees-list">
      {employees.map(emp => (
        <EmployeeCard key={emp.id} employee={emp} />
      ))}
    </div>
  );
}

export default EmployeesList;

```

Step 8: Consume the context in EmployeeCard.js

a. Import ThemeContext and useContext into EmployeeCard.js:

```

import React, { useContext } from 'react';
import ThemeContext from './ThemeContext';

```

b & c. Retrieve the theme value with useContext() and use it for the button's className:

```

function EmployeeCard({ employee }) {
  const theme = useContext(ThemeContext);

  return (
    <div className="employee-card">
      <h3>{employee.name}</h3>
      <p>{employee.designation}</p>
      <button className={theme === 'dark' ? 'btn-dark' : 'btn-light'}>
        View Details
      </button>
    </div>
  );
}

```

```
export default EmployeeCard;
```

Complete EmployeeCard.js (after conversion)

```
import React, { useContext } from 'react';
import ThemeContext from '../ThemeContext';

function EmployeeCard({ employee }) {
  const theme = useContext(ThemeContext);

  return (
    <div className="employee-card">
      <h3>{employee.name}</h3>
      <p>{employee.designation}</p>
      <button className={theme === 'dark' ? 'btn-dark' : 'btn-light'}>
        View Details
      </button>
    </div>
  );
}

export default EmployeeCard;
```

Step 9: Run the application and verify

```
npm start
```

Open a browser window and type the following in the address bar:

```
http://localhost:3000
```

Clicking Toggle Theme in App updates the theme value in state, which flows through ThemeContext.Provider directly to every EmployeeCard via useContext() — without EmployeesList ever needing to know about or forward a theme prop. All employee card buttons should switch styling between light and dark exactly as before, confirming the conversion from props to Context API preserves the original behavior while removing the prop-drilling through EmployeesList.

Result

The employee management application's theme sharing was successfully converted from props to the React Context API. A new ThemeContext.js file defines the context with a default value of 'light'. App.js wraps its JSX in ThemeContext.Provider, supplying the current theme from its state. EmployeesList.js no longer receives or forwards a theme prop. EmployeeCard.js consumes the context directly using useContext(ThemeContext) and applies the corresponding className to its button. The application was verified by running it locally and confirming the theme toggle still works correctly at <http://localhost:3000>.